GenAI and LLM Apps – Practical Perspective

Course Project

Deadline: February 26, 11:59pm ET (8:59pm PT).

In this project, you are required to submit your GitHub Repository URL on Gradescope. A penalty will be applied if you don't do so. If you are working in a team, one submission is sufficient.

General Expectations:

- The course project requires a significant amount of independent, self-directed learning. It is designed to integrate multiple concepts covered throughout lectures, recitations, and assignments. Students should not expect project-specific support during office hours or via Piazza.
- You may choose to work individually or with one peer on this project.
- If you choose to work with a peer, you must:
  - Clearly list your peer's name in your README.md file, and
  - Add your peer to the Gradescope submission.
  - Failure to complete both steps may result in your peer not receiving a grade.
- You must select one of the two available project options.
- Do not change the visibility of your GitHub project repository to public.
- Unlike homework assignments, late project submissions are not accepted. Any project submitted after the deadline on Gradescope or GitHub will receive zero points and will not be graded.
- Not following the submission guidelines, using the appropriate GitHub classroom URL, or complying with any provided logistics will lead to additional penalties.

Option#1: Building a Research Assistant Agent for Paper Recommendations with RAG Pipelines.

Difficulty level: Moderate

Accept this project by accessing GitHub classroom via the following URL: https://classroom.github.com/a/H1My89Vo

Project Overview

In this project, you will design and implement an AI-powered research assistant that combines:

- An AI agent (agentic workflow).
- A Retrieval-Augmented Generation (RAG) pipeline.
- A multilingual translation service.
- A scalable vector database.
- A web-based user interface.

The system should help researchers:

- Retrieve domain-specific information from uploaded academic papers.
- Ask questions in their native language.
- Receive context-aware answers grounded in the indexed papers.
- Obtain recommended research papers for further reading.

System Components

You are required to build and deploy the following components:

1. **AI Agent**
    a. Implemented using LangGraph.
    b. Deployed on Google Kubernetes Engine (GKE).
2. **Translator Service**
    a. Deployed on GKE.
    b. Communicates with the AI agent through a consistent, well-defined interface/system.
3. **Vector Database**
    a. Stores embeddings for 30 academic research papers.

b. Accessible by the AI agent.
c. Deployed on GKE with auto-scaling enabled.

4. **Web Application**
    a. Provides the user interface.
    b. Allows paper upload, querying, and response visualization.

<u>Functional Requirements & Grading Breakdown</u>

- (20%) Web Application (Streamlit)

  Build a web application that allows users to:

  - Paper Upload
    - Upload academic papers for indexing into the RAG system
    - While uploading, users must select the paper's research domain:
      - AI
      - Security
      - Other
  - User Queries
    - Enter a query related to the uploaded papers
    - Queries may be submitted in any of the following languages:
      - English
      - Spanish
      - French
      - Italian
  - System Responses
    - Generate responses using retrieved content from the vector database
    - Return:
      - A direct answer to the user's question
      - A list of two relevant research papers
    - The response must be in the same language as the user's query
      - Example: Spanish input → Spanish output
    - Include a reference list of the two most relevant papers used in the answer

- (30%) AI Agent Design
  - Implement the AI agent using LangGraph.
  - Deploy the agent on GKE.
  - Ensure the agent:
    - Communicates with the translator service via a standardized interface/system and the output format is **always** consistent.
    - Uses a design pattern that minimizes hallucinations.
  - The agent must act as the central orchestrator between:
    - The web app.
    - The translator service.
    - The vector database.
- (15%) Translator Service
  - Build or integrate a translation service that supports only:
    - English
    - Spanish
    - French
    - Italian
  - The service must:
    - Be containerized.
    - Be deployed on GKE.
  - You may:
    - Use a publicly available container, or
    - Build your own Docker image.
  - Helpful reference: https://docs.docker.com/guides/language-translation/
- (20%) RAG Pipeline & Vector Database

  Implement a domain-aware RAG system with the following requirements:

  - Paper Collection: Index 30 academic papers total:
    - 10 AI papers: https://scholar.google.com/scholar?q=Artificial+Intelligence+research+papers
    - 10 Security papers: https://scholar.google.com/scholar?q=security+research+papers

        - 10 papers from other domains: https://scholar.google.com/scholar?q=research+papers

    All papers must be uploaded through your Streamlit application.

  - Embedding & Storage
    - Papers must be:
      - Indexed and embedded in their original language
    - You are free to choose:
      - The vector database technology
      - The collection schema and metadata structure
  - Deployment & Scaling
    - Deploy the vector database on GKE
    - Enable auto-scaling:
      - Minimum: 1 pod
      - Maximum: 5 pods
      - Scaling trigger: CPU utilization ≥ 70%
  - Evaluation & Comparison

    Compare the following indexing algorithms using:
    - Indexing time
    - Total storage size
      - IVF_PQ
      - DiskANN
      - HNSW
    - Use cosine similarity for all similarity calculations
- (10%) System Architecture Diagram
  - Create a clear flowchart illustrating:
    - Component interactions
    - Data flow between services
    - User request lifecycle
- (5%) Limitations & Assumptions
  - Clearly list:
    - System limitations
    - Design assumptions
    - Known trade-offs

General Requirements

- Document all system setup, design decisions, comparisons, experiments, and results in your **README.md**
- Follow best practices discussed in class for:
  - System design
  - Cloud deployment
  - Security
  - Scalability
  - AI agent implementations.
- Not submitting the required video(s) (for both code walkthrough and functionality demo): you will get up to 70% of the maximum grade.
- Not providing clear details in the ReadMe file on how to run the application (or any variables that need to be updated/replaced): you will get up to 90% of the maximum grade.

Final Deliverables

- Source Code
- README Documentation
- System Architecture Diagram
- Video(s):
  - Functional demonstration for a full functionality of the Streamlit app in addition to every service and endpoint deployed on GKE.
  - During your functional demo, send HTTP requests to each service individually, on GKE, to verify that each service is working properly. For example, test the translator service independently, and test the standard communication interface independently.
  - Code walkthrough, explaining every line of code that you wrote in this application.

Hypothetical Example:

(User Query – User selected Spanish as the language and AI as the research domain):

"¿Cuáles son los avances recientes en la búsqueda de arquitecturas neuronales?"

Response:

La búsqueda de arquitecturas neuronales (NAS, por sus siglas en inglés) ha avanzado mediante técnicas como el aprendizaje por refuerzo, métodos basados en gradientes y algoritmos evolutivos. Los enfoques recientes incluyen NAS diferenciable para una optimización más eficiente.

Artículos de Investigación Recomendados:

- DARTS: Differentiable Architecture Search. URL: http://abc123
- Efficient Neural Architecture Search via Parameter Sharing. URL: http://abc456

Option#2: Multimodal AI Agent for Augmenting Metaverse Environments

Difficulty level: Challenging

Accept this project by accessing GitHub classroom via the following URL: https://classroom.github.com/a/HDNkeX6y

In this project, you will design and deploy a multimodal AI agent that interacts with users through prompts and dynamically creates, modifies, and renders new scenes inside an open-source Metaverse environment.

The goal is to demonstrate how AI-driven multimodal reasoning (text + image understanding/generation) can be integrated into an existing Metaverse platform to enhance immersion, interactivity, and world-building capabilities.

High-Level Objective

- Your system should allow a user to:
  - Provide a prompt (e.g., describing a scene, object, or environment change).
  - Have the AI agent interpret the prompt.
  - Automatically generate or modify a scene.
  - Reflect the changes live within the Metaverse world.
  - The newly generated scene must be merged into an existing Metaverse environment and rendered through the Metaverse server.

Metaverse Platform Requirements

- Use Vircadia Metaverse via this URL: https://github.com/vircadia/
- You may also use:
  - Vircadia World
  - Vircadia World SDK

System & Deployment Requirements

- Your solution must satisfy the following:
  - (40%) AI Agent
    - Implements multimodal reasoning (e.g., text → scene generation, image understanding, or image generation)
    - Accepts user prompts and determines how the Metaverse should be updated
  - (30%) Metaverse Server
    - Deployed on Google Kubernetes Engine (GKE)
    - Fully scalable with auto-scaling:
      - Minimum: 1 pod
      - Maximum: 5 pods
      - Scaling trigger: CPU utilization ≥ 70%
    - Capable of rendering and updating scenes dynamically
  - (30%) Integration
    - The Metaverse must be extended to:
      - Accept user prompts
      - Apply AI-generated outputs
      - Render newly created or modified scenes in real time
    - The AI agent and Metaverse server must communicate in a clean, well-defined manner
- Output Expectations
  - A sample demo video is provided on the Canvas project page
  - Replicating the demo behavior (or an equivalent experience) using a deployed Metaverse server will result in full credit for this project option.
  - Creativity and technical ambition are encouraged but not required beyond the demo baseline and the provided technical requirements.

Grading Policy

- Grading for this option is subjective
- Emphasis is placed on:
  - Successful end-to-end integration
  - Functional scene generation and rendering

    - Stability, scalability and deployment correctness
    - Video submission is mandatory
    - Failure to submit a video will result in zero points for this project option
- Required Deliverables
    - Deployed AI Agent on GKE
    - Deployed and Scalable Metaverse Server on GKE
    - Working Prompt-to-Scene Pipeline
    - Recorded Video Demonstration
        - Show user prompting
        - Show AI-driven scene creation
        - Show the updated Metaverse environment

Helpful Resources

- You may find the following resources useful when implementing multimodal capabilities:
    - Multimodal image understanding: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/image-understanding
    - Image generation overview: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/image/overview
    - Imagen + Vertex AI tutorial: https://lablab.ai/t/imagen-vertexai-tutorial

Notes & Recommendations

- This option is technically demanding and best suited for:
    - Students that are comfortable with:
        - Distributed systems
        - Kubernetes-based deployments
        - Multimodal AI workflows
- Strong system design and a clean demo can outweigh incomplete advanced features.